DSA0109 – Assignment 1 (CO1)

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

This solution follows your uploaded assignment PDF. It includes the written answers, pseudocode, C++ programs, predicted outputs, testing tables, Big-O analysis, and the Day-4 design challenge.

Important:

Compile and run the programs yourself, save your actual output screenshots, and add your own GitHub repository link. The assignment specifically requires evidence of your own execution and progressive commits.

Scenario and assumptions

The original metro system has three zones and the following rules:

Rule	Value
Inner zone	₹10
Middle zone	₹20
Outer zone	₹30
Peak surcharge	20%
Peak hours	8:00–10:00 AM and 5:00–8:00 PM
Minimum balance after travel	₹50
Money representation	Integer paise

The peak intervals are 8:00 inclusive to 10:00 exclusive, and 17:00 inclusive to 20:00 exclusive. Thus, 10:00 AM and 8:00 PM are off-peak.

For the original assignment, student and senior-citizen concessions are mentioned but their percentages are not specified. I will keep the base implementation focused on the defined fares and surcharge, and state where the concession rule needs an agreed percentage. The new Airport zone, weekend discount, and monthly pass are handled in Task 12.

Task 1 – Understand the Problem and Identify the Objects (4 marks)

1.1 Objects, data and behaviour

Object	Data	Behaviour
SmartCard	Card ID, balance in paise	Check balance, recharge, deduct fare
Passenger	Passenger ID, passenger category	Identify student, senior citizen or regular passenger
Journey	Starting zone, destination/farthest zone, time	Identify applicable zone and peak period
Zone	Zone name	Represent Inner, Middle or Outer
FareTable	Base fares, surcharge percentage	Calculate the correct fare

Each object represents a real part of the metro system. For example, SmartCard controls its own balance so other parts of the program cannot accidentally make it negative.

1.2 Procedural vs object-oriented programming

Procedural approach: Use separate variables and functions such as balance, calculateFare() and canTravel(). The data and functions are separate.

Object-oriented approach: Keep data and related functions together in classes. For example, SmartCard contains the balance and provides functions to check and deduct it.

The object-oriented approach is easier to maintain when the metro adds new rules.

1.3 Four principles of OOP

- Encapsulation: Keep the card balance private and modify it only through class functions.
- Abstraction: Provide calculateFare() without exposing every internal calculation.
- Inheritance: A StudentPassenger or SeniorPassenger could inherit from a common Passenger class.
- Polymorphism: Different passenger types could implement a virtual getDiscount() function.

Inheritance and polymorphism are optional in the first version; they become useful when passenger categories have different fare rules.

1.4 Correctness rules

1. Fare must never be negative.
2. Money calculations must be exact.

3. Peak hours must follow the defined time boundaries.
4. A journey is allowed only if the remaining balance is at least ₹50.
5. An invalid zone or input must be rejected.
6. A card balance must never become negative.
7. Every journey must use the same centrally defined fare rules.

1.5 Alternative concept and complexity

Both procedural and object-oriented versions can calculate one fare in \(O(1)\) time. Both use \(O(1)\) extra space per card.

The main difference is maintainability. With global variables, many functions may need changes when a rule changes. With classes, the rule can usually be updated in one central method.

Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 marks)

2.1 Five kinds of C++ tokens

Token type	Example	Meaning
Keyword	int, class, return	Reserved C++ word
Identifier	balance, fare	Programmer-defined name
Constant	5000, 'A'	Fixed value
Operator	+, -, >=	Performs an operation
Punctuator	{}, ;, ()	Structures the program

2.2 Define constants in one place

```
constexpr long long INNER_FARE_PAISE = 1000;
constexpr long long MIDDLE_FARE_PAISE = 2000;
constexpr long long OUTER_FARE_PAISE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

- #define performs text substitution and does not provide normal type checking or scope.
- const declares a value that cannot be modified.
- constexpr allows compile-time evaluation when possible and is suitable for fixed fare values.
- enum class provides a strongly typed set of named values.
- enum class is safer than a traditional enum because its values do not implicitly convert to integers and its enumerators are scoped.

Naming convention:
Use descriptive names. Constants use uppercase with underscores; variables and functions use meaningful names such as `balancePaise` and `calculateFare()`.

-

Alternative concept and complexity

- Replacing `constexpr` values with macros changes type safety and scope, not the basic fare calculation. Both approaches generally require \(O(1)\) time. A macro has no separate storage, and a `constexpr` value also often requires no runtime storage, although storage can be required in some circumstances.

Task 3 – Choose Data Types and Check Type Compatibility (4 marks)

3.1 Data type selection

Value	Recommended type	Reason
Fare in paise	`long long`	Exact integer money
Card balance	`long long`	Supports large balances
Zone	`enum class Zone`	Named, type-safe values
Hour	`int`	Stores 0–23
Peak status	`bool`	True or false
Passenger category	`enum class`	Represents valid categories
Daily journey count	`long long`	Handles millions of journeys
Character	`char`	Stores a character
Approximate scientific measurement	`double`	Supports fractional values
Type inferred from an expression	`auto`	Compiler determines the type

3.2 Why use integer paise?

- ₹10.25 is represented as `1025` paise. Integer arithmetic avoids floating-point rounding errors.

```
long long farePaise = 1025;
long long balancePaise = 10000;
balancePaise -= farePaise;
```

- The remaining balance is exactly 8975 paise, or ₹89.75.
- Using double for rupees can produce small representation errors because many decimal fractions cannot be represented exactly in binary floating point.

3.3 Type sizes and ranges

- The exact sizes must be confirmed on the student's compiler. On a typical 64-bit GCC system:

Type	Typical size	Approximate range
`bool`	1 byte	`false`, `true`
`char`	1 byte	Usually −128 to 127 or 0 to 255
`int`	4 bytes	−2,147,483,648 to 2,147,483,647
`unsigned int`	4 bytes	0 to 4,294,967,295
`long long`	8 bytes	Approximately $\pm 9.22 \times 10^{18}$
`float`	4 bytes	Approximately 6–7 significant decimal digits
`double`	8 bytes	Approximately 15–16 significant decimal digits

C++

```
#include <climits>
#include <iostream>
using namespace std;

int main() {
    cout << "int: " << sizeof(int) << " bytes\n";
    cout << "long long: " << sizeof(long long) << " bytes\n";
    cout << "double: " << sizeof(double) << " bytes\n";
    cout << "INT_MAX: " << INT_MAX << '\n';
    cout << "LLONG_MAX: " << LLONG_MAX << '\n';
}
```

3.4 Daily collection and overflow

Assume all 3,000,000 journeys cost ₹30 before any surcharge.

\[3{,}000{,}000 \times ₹30 = ₹90{,}000{,}000 \]

That is ₹9 crore, or 9,000,000,000 paise.

The paise total exceeds INT_MAX (2,147,483,647). Therefore, a 32-bit int cannot safely store this total in paise. Use long long.

Alternative: double vs long long

Both generally take 8 bytes and have \(O(1)\) addition time on common systems. But long long stores integer paise exactly within its range, whereas double can introduce rounding error.

To compare the types experimentally, accumulate the same list of fares using both representations and print the difference. Do not assume a particular measured error without running the experiment.

Task 4 – Control Implicit Conversions (4 marks)

4.1 Important conversions

- int / int performs integer division.
- Assigning the result of integer division to double does not restore the discarded fraction.
- A char used in arithmetic is promoted to int.
- A nonzero integer converts to true; zero converts to false.
- Comparing signed and unsigned integers can convert the signed operand to unsigned.
- Brace initialization rejects narrowing conversions such as int x{3.7};.
- static_cast<double>(a) explicitly converts a to double.

4.2 Predict the output

Program from the assignment:

```
#include <iostream>


int main() {
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;


    std::cout << c << ' ' << d << '\n';


    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';


    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';


    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Expected output

```
3 3.5
C 68
1 true
s >= u
```

4.3 Explanation

1. a / b is integer division: 7 / 2 becomes 3. Assigning it to double produces 3.0, printed as 3 by default.
2. static_cast<double>(a) / b produces 3.5.
3. 'A' + 2 produces the character 'C' on a system using the usual ASCII-compatible character encoding.
4. ch + 1 promotes ch to int; 'C' has the numeric value 67 in ASCII, so the result is 68.
5. The integer 5 converts to true. Normal bool output is 1; std::boolalpha prints true.
6. The signed value -1 converts to an unsigned value during the comparison. It becomes a very large unsigned value, so s < u is false.

.4 Safer coding

```
int whole{3};

// int wrong{3.7};  // Compilation error: narrowing


double exact = static_cast<double>(7) / 2;
```

Alternative concept and complexity

Explicit conversions improve clarity and can expose mistakes at compile time. They generally do not change asymptotic time or memory. However, an explicit cast does not automatically make an unsafe conversion correct; the programmer must still check the range and intended meaning.

Task 5 – Write Correct Expressions with the Right Precedence (4 marks)

5.1 Peak-hour expression

```
bool peak =
   (hour >= 8 && hour < 10) ||
   (hour >= 17 && hour < 20);
```

But the first version is easier to read.

5.2 Surcharge and concession

The surcharge can be calculated with integer arithmetic:

fare = fare + fare * 20 / 100;

For ₹30, represented as 3000 paise:

$$ 3000 + \frac{3000 \times 20}{100} = 3600 $$

The result is 3600 paise, or ₹36.

For a concession percentage defined by the system, use a separate rule:

fare = fare - fare * discountPercent / 100;

The assignment does not specify the student or senior discount percentages. The implementation must use a policy-approved percentage rather than inventing one.

5.3 Other operators

- x++: uses the old value, then increments.
- ++x: increments first, then uses the new value.
- i = i++ should be avoided. In modern C++, this particular expression is defined, but it assigns the old value back to i, which makes the intended increment ineffective.
- Integer division truncates toward zero: -10 / 3 is -3.
- The remainder follows the dividend's sign: -10 % 3 is -1.
- Compound assignment, such as balance -= fare, updates the left operand.
- The conditional operator condition ? a : b chooses one of two expressions.

5.4 Predict the output

```
#include <iostream>


int main() {
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';


    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';
```

```
    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' '
              << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3)
        k += i;
    std::cout << k << '\n';
}
```

Expected output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

5.5 Explanation of each line

1. Multiplication happens before addition: 5 + 3 * 2 = 11; parentheses force (5 + 3) * 2 = 16.
2. x++ contributes 5 and then increments x to 6. ++y increments y from 3 to 4 before contributing 4. Thus, z = 9.
3. x > y is true, so the conditional expression prints 6. Then % is evaluated before *: 6 % 4 * 2 = 4.
4. The division and remainder results follow integer arithmetic and truncation toward zero.
5. The loop adds 0 + 3 + 6 + 9 = 18.

Alternative: lookup table for peak hours

A bool peak[24] array can store whether each hour is peak. A lookup then takes $O(1)$ time, just like the expression, but needs approximately 24 bytes if each bool occupies one byte. The expression avoids the table and is adequate for this small rule set.

Task 6 – Design the Control Structures (4 marks)

6.1 Which control structures should be used?

Structure	Use in the metro system
if–else	Check balance and peak status
switch	Select the base fare for a zone
Conditional operator	Choose between two simple values
for loop	Process a known number of journeys
Range-based for	Process every trip in an array or collection
while loop	Continue until the operator chooses to stop
break	Exit a loop or switch
continue	Skip an invalid journey

For a known list of journeys, a range-based for loop is readable and convenient.

6.2 Pseudocode

```
BEGIN
   SET inner fare to 1000 paise
   SET middle fare to 2000 paise
   SET outer fare to 3000 paise
   SET surcharge to 20 percent
   SET minimum balance to 5000 paise

   READ zone, peak status and card balance

   SELECT zone
      CASE Inner: SET fare to 1000
      CASE Middle: SET fare to 2000
      CASE Outer: SET fare to 3000
      DEFAULT: REPORT invalid zone and STOP
   END SELECT
```

```
    IF peak is true
        ADD 20 percent of fare using integer arithmetic
    END IF

    IF balance minus fare is at least 5000 paise
        DEDUCT fare
        DISPLAY fare and remaining balance
    ELSE
        DISPLAY journey denied
    END IF
END
```

6.3 Completed C++ program

Save as Task06.cpp.

```
#include <iomanip>
#include <iostream>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;

    switch (z) {
```

```
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
        default:
            return -1;
    }

    if (peak)
        base += base * PEAK_SURCHARGE_PERCENT / 100;

    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;

    struct Trip {
        Zone zone;
```

```
    bool peak;
};

Trip trips[] = {
    {Zone::Inner, false},
    {Zone::Outer, true},
    {Zone::Middle, false}
};

for (const Trip& t : trips) {
    long long f = fare(t.zone, t.peak);

    if (canTravel(balance, f)) {
        balance -= f;
        cout << "Fare Rs " << f / 100 << '.'
             << setw(2) << setfill('0') << f % 100
             << setfill(' ');
    } else {
        cout << "Denied (low balance)";
    }

    cout << " balance Rs " << balance / 100 << '\n';
}
}
```

6.4 Expected output

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

6.2 Pseudocode

```
BEGIN
   SET inner fare to 1000 paise
   SET middle fare to 2000 paise
   SET outer fare to 3000 paise
   SET surcharge to 20 percent
   SET minimum balance to 5000 paise

   READ zone, peak status and card balance

   SELECT zone
      CASE Inner: SET fare to 1000
      CASE Middle: SET fare to 2000
      CASE Outer: SET fare to 3000
      DEFAULT: REPORT invalid zone and STOP
   END SELECT

   IF peak is true
      ADD 20 percent of fare using integer arithmetic
   END IF

   IF balance minus fare is at least 5000 paise
      DEDUCT fare
      DISPLAY fare and remaining balance
   ELSE
      DISPLAY journey denied
   END IF
END
```

6.3 Completed C++ program

Save as Task06.cpp.

```
#include <iomanip>
#include <iostream>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;

    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
        default:
            return -1;
    }

    if (peak)
```

```
        base += base * PEAK_SURCHARGE_PERCENT / 100;

    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;

    struct Trip {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {
            balance -= f;
```

```
            cout << "Fare Rs " << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ');
        } else {
            cout << "Denied (low balance)";
        }

        cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

6.4 Expected output

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Why is the third journey denied? After the first two journeys, the balance is ₹54. Paying ₹20 would leave ₹34, which is below the required ₹50.

6.5 Test 10 AM and 8 PM

Use this peak rule:

```
bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}
```

Time	Peak?	Outer-zone fare
08:00	Yes	₹36
09:00	Yes	₹36
10:00	No	₹30
17:00	Yes	₹36
19:00	Yes	₹36

20:00 No ₹30

Alternative: array vs switch

An array lookup and a switch are both \(O(1)\) for this fixed set of zones. The array occupies a small amount of storage. A fourth zone requires adding a new array element and enum value, or adding another switch case. The array becomes convenient when fares are data-driven, but the program must validate the index.

Task 7 – From Procedural to Object-Oriented (4 marks)

7.1 Procedural version

Save as Task07_procedural.cpp.

```
#include <iostream>

using namespace std;


constexpr long long MIN_BALANCE = 5000;


bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance >= fare &&
           balance - fare >= MIN_BALANCE;
}


int main() {
    long long balance = 10000;
    long long fare = 3000;


    if (canTravel(balance, fare)) {
        balance -= fare;
        cout << "Journey allowed\n";
    } else {
        cout << "Journey denied\n";
    }
```

```
    cout << "Balance in paise: " << balance << '\n';
}
```

Expected output:

Journey allowed

Balance in paise: 7000

The balance is kept separately from the function. Other functions could accidentally change it incorrectly.

7.2 Object-oriented version

Save as Task07_oop.cpp.

```
#include <iostream>
using namespace std;

class SmartCard {
private:
    long long balance;
    static constexpr long long MIN_BALANCE = 5000;

public:
    explicit SmartCard(long long initialBalance)
        : balance(initialBalance >= 0 ? initialBalance : 0) {}

    bool canTravel(long long fare) const {
        return fare >= 0 && balance >= fare &&
               balance - fare >= MIN_BALANCE;
    }

    bool deduct(long long fare) {
        if (!canTravel(fare))
            return false;
```

```
        balance -= fare;
        return true;
    }

    long long getBalance() const {
        return balance;
    }
};

int main() {
    SmartCard card(10000);

    if (card.deduct(3000))
        cout << "Journey allowed\n";
    else
        cout << "Journey denied\n";

    cout << "Balance in paise: "
         << card.getBalance() << '\n';
}
```

Expected output:

Journey allowed

Balance in paise: 7000

7.3 Comparison

Feature	Procedural	OOP
Balance storage	Separate variable	Private member
Data protection	Depends on programmer	Encapsulation
Rule enforcement	Every caller must follow rules	Centralized in SmartCard
Changing minimum balance	Update shared rule and affected code	Update class constant
Time per deduction	\(O(1)\)	\(O(1)\)
Space per card	\(O(1)\)	\(O(1)\)

A public balance would not change time or space complexity, but any part of the program could change it to a negative value or bypass the minimum-balance rule. A private member with controlled methods is safer.

Validation note: The constructor in this example prevents a negative starting balance by replacing it with zero. A production application should preferably reject invalid initial balances explicitly instead of silently changing them.

Task 8 – Read and Print Data Safely (4 marks)

8.1 Safe input handling

The following program reads an hour and recovers when the operator types letters instead of a number.

Save as Task08.cpp.

```
#include <iomanip>

#include <iostream>

#include <limits>

using namespace std;


int main() {

    int hour;


    while (true) {
```

```
        cout << "Enter hour (0-23): ";

        if (cin >> hour && hour >= 0 && hour <= 23)
            break;

        cout << "Invalid input. Try again.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    bool peak = (hour >= 8 && hour < 10) ||
                (hour >= 17 && hour < 20);

    cout << boolalpha << "Peak: " << peak << '\n';

    long long farePaise = peak ? 3600 : 3000;

    cout << fixed << setprecision(2);
    cout << "Fare Rs "
         << static_cast<double>(farePaise) / 100.0 << '\n';
}
```

Example input:

Enter hour (0-23): abc

Invalid input. Try again.

Enter hour (0-23): 18

Expected remaining output:

Peak: true

Fare Rs 36.00

The fare is stored internally in integer paise. Floating point is used here only for formatted display; an exact paise-formatting function can also print the rupee and paise parts separately.

8.2 Why these input/output functions matter

- cin.fail() or the failure state of cin detects invalid input.
- cin.clear() clears the error state.
- cin.ignore() discards the bad input.
- setw(2) sets the width of the next output field.
- setfill('0') pads the field with zeros.
- fixed and setprecision(2) display two decimal places.

Alternative: getline() and stoi()

getline() reads a whole line, and stoi() converts it to an integer. A try/catch block can handle invalid values. Processing a line of length \(L\) takes \(O(L)\) time and \(O(L)\) buffer space. Direct cin >> is convenient for numeric input, but it also requires careful recovery and range validation.

Task 9 – Implement and Test the Fare Logic (4 marks)

9.1 Complete fare logic

The following is a reusable implementation for the original three-zone system. It checks invalid zones, applies the peak surcharge, and preserves the minimum balance.

Save as Task09.cpp.

```
#include <iostream>

using namespace std;


enum class Zone { Inner, Middle, Outer, Invalid };


constexpr long long MIN_BALANCE = 5000;

constexpr int SURCHARGE = 20;


bool isPeak(int hour) {

   return hour >= 0 && hour < 24 &&

         ((hour >= 8 && hour < 10) ||

          (hour >= 17 && hour < 20));
```

```
}

long long calculateFare(Zone zone, int hour) {
    long long base;

    switch (zone) {
        case Zone::Inner:  base = 1000; break;
        case Zone::Middle: base = 2000; break;
        case Zone::Outer:  base = 3000; break;
        default: return -1;
    }

    if (hour < 0 || hour > 23)
        return -1;

    if (isPeak(hour))
        base += base * SURCHARGE / 100;

    return base;
}

bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance >= fare &&
           balance - fare >= MIN_BALANCE;
}

int main() {
    long long balance = 10000;
```

```
struct Test {
    Zone zone;
    int hour;
};

Test tests[] = {
    {Zone::Inner, 12},
    {Zone::Middle, 9},
    {Zone::Outer, 14},
    {Zone::Outer, 18},
    {Zone::Inner, 20}
};

long long totalFare = 0;

for (const Test& t : tests) {
    long long f = calculateFare(t.zone, t.hour);

    if (canTravel(balance, f)) {
        balance -= f;
        totalFare += f;
        cout << "Fare paise: " << f
             << ", balance paise: " << balance << '\n';
    } else {
        cout << "Journey denied or invalid\n";
    }
}

cout << "Total collected paise: " << totalFare << '\n';
```

}

9.2 Hand-calculated test cases

Test	Zone	Time	Fare	Expected decision
1	Inner	12:00	₹10	Allow if balance permits
2	Middle	09:00	₹24	Allow if balance permits
3	Outer	14:00	₹30	Allow if balance permits
4	Outer	18:00	₹36	Allow if balance permits
5	Inner	20:00	₹10	Allow if balance permits

Total before balance restrictions:

\[₹10 + ₹24 + ₹30 + ₹36 + ₹10 = ₹110 \]

The example starts with ₹100 and requires at least ₹50 to remain. Therefore, not all five journeys can be accepted. The actual collected total will be lower than ₹110 because the system must deny journeys that violate the balance rule.

9.3 Boundary and invalid-input tests

Test	Input	Expected result
Boundary	08:00	Peak
Boundary	10:00	Off-peak
Boundary	17:00	Peak
Boundary	20:00	Off-peak
Balance	₹60, fare ₹10	Allow; exactly ₹50 remains
Balance	₹59.99, fare ₹10	Deny
Invalid zone	Unknown zone	Reject
Invalid recharge	−₹100	Reject
Invalid hour	Letters instead of a number	Recover and ask again

The program above demonstrates fare calculation and balance checks. A production-ready version must also validate the zone and recharge when reading operator input. The Zone::Invalid value is a simple way to represent an invalid zone internally.

Alternative: test-case table

Using a table of inputs and expected outputs makes tests easier to expand. Running \(t\) test cases takes \(O(t)\) time. The test table uses \(O(t)\) space. A new boundary test can be added as one more row instead of creating another separate test block.

Task 10 – Compare Expectations with Execution (4 marks)

This task asks you to compare floating-point and integer-paise totals, check overflow, and record compiler warnings. The exact measurements depend on the actual fare dataset and compiler, so the following gives a reproducible experiment rather than claiming that it has already been run on your machine.

10.1 Journey-volume calculations

Assume, for a simple upper-bound example, that every journey costs ₹30, with no surcharge.

Journeys	Total rupees	Total paise	Fits in 32-bit signed int paise?
20,000	₹600,000	60,000,000	Yes
500,000	₹15,000,000	1,500,000,000	Yes
3,000,000	₹90,000,000	9,000,000,000	No

The 32-bit signed int maximum is 2,147,483,647. The whole-network total exceeds it.

The actual total depends on the distribution of zones, peak journeys and discounts. The table assumes all journeys cost ₹30, so it is a scenario calculation, not measured metro data.

10.2 C++ experiment: integer paise vs double

Save as Task10.cpp.

```
#include <iomanip>
#include <iostream>
using namespace std;

int main() {
    const int n = 3000000;
    long long totalPaise = 0;
    double totalRupees = 0.0;

    for (int i = 0; i < n; ++i) {
        totalPaise += 3000;
```

```
        totalRupees += 30.0;
    }


    cout << "Integer paise: " << totalPaise << '\n';
    cout << fixed << setprecision(2);
    cout << "Double rupees: " << totalRupees << '\n';
    cout << "Integer total in rupees: "
         << static_cast<double>(totalPaise) / 100.0 << '\n';
}
```

Expected numerical totals:

Integer paise: 9000000000

Double rupees: 90000000.00

Integer total in rupees: 90000000.00

For this particular experiment, the repeated addition of 30.0 is exact in ordinary binary floating point because 30 is exactly representable. Other decimal fares may not be exact. Compare the totals using your actual fare dataset.

10.3 Compiler warnings

Compile each program using:

g++ -std=c++17 -Wall -Wextra Task10.cpp -o Task10

-Wall -Wextra enables additional warnings. The compiler's actual warning output should be copied into your report. Typical issues include unused variables, signed/unsigned comparisons, or missing return paths. The sample program above may compile without warnings; do not invent warnings that you did not receive.

10.4 Type comparison

Type	Addition	Space	Main risk
int	\(O(1)\)	Usually 4 bytes	Signed overflow
long long	\(O(1)\)	Usually 8 bytes	Overflow beyond its range
double	\(O(1)\)	Usually 8 bytes	Approximation of some decimal values

Signed integer overflow in C++ is undefined behaviour. Do not rely on it wrapping around. Use a sufficiently large integer type and check the possible maximum total.

Task 11 – Recommend and Defend the Final Design (4 marks)

11.1 Recommended design

Use:

- enum class Zone to represent zones.
- constexpr constants for fixed fares and percentages.
- long long to store money in paise and accumulate large totals.
- SmartCard to protect the card balance.
- FareTable or a central fare-calculation function to avoid repeating fare values.
- A separate peak-hour function with explicit boundaries.
- Input validation for zone, hour, recharge and card balance.
- Automated tests for normal journeys, boundary times and low balances.

11.2 Changing a fare

If the Inner fare changes from ₹10 to ₹12, update the corresponding constant or fare-table entry. The same fare calculation method should then use the revised value everywhere.

This is safer than typing 1000 in many different functions.

11.3 Limitations

The first design has some limitations:

1. It supports only three zones.
2. It does not define the exact student or senior discount percentages.
3. It does not support weekend discounts or passes.
4. A simple in-memory card balance does not provide transaction persistence or concurrent-update protection.
5. It needs additional validation for operator input and real-world transaction failures.

11.4 Does it scale to the whole network?

The fare calculation and balance check take \(O(1)\) time per journey. Processing \(n\) journeys takes \(O(n)\) total time. The fare calculation itself uses \(O(1)\) extra space per journey, excluding stored trip history and other system infrastructure.

Three million journeys per day do not change the Big-O complexity, but the implementation must use suitable numeric ranges, reliable transaction handling, and performance tests.

Alternative: read the fare table from a configuration file

If there are \(z\) zones, reading a fare table at startup takes \(O(z)\) time and \(O(z)\) storage. A configuration file allows fare changes without recompiling the program. However, the program must validate the file, handle missing or invalid values, and ensure that all gates use the same approved configuration. For only three fixed fares, compiled constants are simpler.

Task 12 – Re-evaluate and Adapt the Design (8 marks)

The metro introduces three new requirements:

1. A fourth zone, Airport, with a base fare of ₹60.
2. A 10% weekend discount that does not combine with the peak surcharge.
3. A monthly pass allowing unlimited journeys in two chosen zones.

A. Impact analysis (2 marks)

Component	Required change
Zone enum	Add Airport
Fare table	Add 6000 paise
Peak surcharge	Keep 20%
Weekend discount	Add a 10% rule
SmartCard	Keep balance private
Pass class	Add pass validity and two chosen zones
Fare calculation	Handle zone, peak time and weekend status
Tests	Add Airport, weekend and pass cases

The original switch can support Airport by adding another case. However, as the number of zones and fare rules grows, a fare table plus separate discount logic is easier to maintain.

B. Evaluate the original design (2 marks)

The original design is correct for its three-zone requirements, but it needs modification.

Worked examples:

- Inner, off-peak weekday: ₹10.
- Inner, peak weekday: ₹10 + 20% = ₹12.
- Airport, off-peak weekday: ₹60.
- Airport, peak weekday: ₹60 + 20% = ₹72.
- Airport, weekend: ₹60 − 10% = ₹54.

The weekend discount must not be combined with the peak surcharge. Therefore, a weekend peak journey costs ₹54 under the stated rule, not ₹64.80.

A monthly pass changes the payment logic because a covered journey should not deduct the normal fare. It needs its own validity and zone checks.

C. Revised implementation (2 marks)

Save as Task12.cpp.

```
#include <array>
#include <iostream>
using namespace std;

enum class Zone { Inner, Middle, Outer, Airport };

constexpr array<long long, 4> FARES = {
    1000, 2000, 3000, 6000
};

constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_PERCENT = 10;
constexpr long long MIN_BALANCE = 5000;

bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

long long calculateFare(Zone zone, int hour, bool weekend) {
    int index = static_cast<int>(zone);

    if (index < 0 || index >= static_cast<int>(FARES.size()) ||
        hour < 0 || hour > 23)
```

```
        return -1;

    long long fare = FARES[index];

    // Weekend discount replaces the peak surcharge.
    if (weekend) {
        fare -= fare * WEEKEND_PERCENT / 100;
    } else if (isPeak(hour)) {
        fare += fare * PEAK_PERCENT / 100;
    }

    return fare;
}

class SmartCard {
private:
    long long balance;

public:
    explicit SmartCard(long long initial)
        : balance(initial >= 0 ? initial : 0) {}

    bool canTravel(long long fare) const {
        return fare >= 0 && balance >= fare &&
               balance - fare >= MIN_BALANCE;
    }

    bool deduct(long long fare) {
        if (!canTravel(fare))
```

```
            return false;

        balance -= fare;
        return true;
    }

    long long getBalance() const {
        return balance;
    }
};

class Pass {
private:
    array<Zone, 2> zones;
    bool active;

public:
    Pass(Zone first, Zone second)
        : zones{first, second}, active(true) {}

    bool covers(Zone zone) const {
        return active &&
               (zone == zones[0] || zone == zones[1]);
    }

    void deactivate() {
        active = false;
    }
};
```

```
int main() {
    SmartCard card(20000);  // ₹200
    Pass pass(Zone::Inner, Zone::Middle);

    struct Trip {
        Zone zone;
        int hour;
        bool weekend;
    };

    Trip trips[] = {
        {Zone::Inner, 9, false},
        {Zone::Airport, 18, false},
        {Zone::Airport, 18, true},
        {Zone::Middle, 12, false}
    };

    for (const Trip& trip : trips) {
        if (pass.covers(trip.zone)) {
            cout << "Monthly pass used\n";
            continue;
        }

        long long fare = calculateFare(
            trip.zone, trip.hour, trip.weekend);

        if (card.deduct(fare)) {
            cout << "Fare paise: " << fare
```

```
            << ", balance paise: "
            << card.getBalance() << '\n';
        } else {
            cout << "Invalid journey or insufficient balance\n";
        }
    }
}
```

Expected output

Monthly pass used

Fare paise: 7200, balance paise: 12800

Fare paise: 5400, balance paise: 7400

Monthly pass used

The first and fourth journeys use the monthly pass. The second is an Airport peak weekday journey, costing ₹72. The third is an Airport weekend journey, costing ₹54.

Implementation note: This is a classroom implementation. A real monthly pass would also need an expiry date, purchase/payment details, and validation that the passenger's pass is active on the journey date. The example represents validity with a simple Boolean.

D. Evidence-based defence (2 marks)

The revised design is preferable because it separates fare data, fare rules, card balance management and monthly-pass validation.

Criterion	**Original design**	**Revised design**
Three zones	Supported	Supported
Airport zone	Requires switch modification	Add fare-table entry
Weekend discount	Not supported	Supported
No combined discounts	Not supported	Explicit rule order
Monthly pass	Not supported	Separate Pass class
Fare lookup	\(O(1)\)	\(O(1)\)
Space per fare table	Small fixed table	Four fare entries

The fare calculation remains $O(1)$, and processing n journeys remains $O(n)$. The revised design adds a small amount of storage but is easier to extend.

For your report, run the tests, record the actual results, and compare them with the expected outputs. Do not claim measured performance or test results unless you have executed the programs.